系统架构综合报告

生成时间: 2026-03-29 **版本**: v1.0 **文档类型**: 技术架构综述

目录

- [HARNESS架构 - 工业级Agent执行框架](#1-harness架构---工业级agent执行框架)
- [DEERFLOW机制 - 复杂任务处理框架](#2-deerflow机制---复杂任务处理框架)
- [OPENVIKING记忆系统 - 分层记忆架构](#3-openviking记忆系统---分层记忆架构)
- [任务完成全链条机制](#4-任务完成全链条机制)
- [已打通的渠道集成](#5-已打通的渠道集成)

---

1. HARNESS架构 - 工业级Agent执行框架

1.1 概述

HARNESS（Human-Assured Reliable Agent Navigation & Execution Support System）是一个工业级Agent执行框架，其核心理念是**"人类掌舵，Agent执行"**。该框架通过标准化工作流程，确保AI Agent在复杂任务中的可靠性、可追溯性和可控性。

1.2 核心理念

原则	描述
人类掌舵	关键决策点由人类确认，Agent负责执行
标准流程	10步标准化工作流确保一致性
全程可追溯	每个动作都有日志记录，支持审计
异常熔断	检测到异常时自动暂停等待人工干预

1.3 10步标准工作流程

```
`
HARNESS 10 步 标 准 工 作 流 程
① 任务
```

```
接入 ──→ ② 意图解析 ──→ ③ 风险评估 ──→ ④ 方案生成 │ │ │ │ │ │ │ ▼ ▼ ▼ ▼ │ │ ⑤ 人工确认 ──→ ⑥ 任务分解 ──→ ⑦ 并行执行 ──→ ⑧ 结果聚合 │ │ │ │ │ │ │ ▼ ▼ ▼ ▼ │ │ ⑨ 质量校验 ──→ ⑩ 数据沉淀 │ │ │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

详细步骤说明

步骤	名称	功能描述	关键输出
①	任务接入	接收用户输入，解析原始需求	任务描述、优先级标签
②	意图解析	使用LLM理解用户真实意图	意图分类、提取实体
③	风险评估	评估任务风险等级	风险评分、风险类型
④	方案生成	制定执行计划和备选方案	执行方案、资源需求
⑤	人工确认	高风险操作需人工审核	确认/驳回指令
⑥	任务分解	将复杂任务拆解为子任务	子任务列表、依赖关系
⑦	并行执行	多Agent并行处理子任务	执行日志、中间结果
⑧	结果聚合	合并子任务结果	完整输出、一致性校验
⑨	质量校验	自动检查输出质量	质检报告、修正建议
⑩	数据沉淀	保存至记忆系统	结构化数据、经验总结

1.4 与OpenClaw的对应关系

HARNESS概念	OpenClaw实现	说明
Agent	Subagent / Main Agent	主从架构，支持并行
任务队列	Session Management	基于channel的任务调度
风险评估	TOOLS.md Policy	文件级权限控制
人工确认	Interactive Mode	`--ask` 参数控制
记忆系统	Memory Files	`memory/` 目录结构
执行沙箱	Docker Sandbox	隔离执行环境
日志追踪	Session Logs	自动记录所有操作

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ OpenClaw
↔ HARNESS 映 射 │
```

```
├──────────────────────────────────────────────────────────────────────────────────────────┤ │ │ │ HARNESS
Layer  OpenClaw  Layer │ │ ──────────────── ──────────────── │ │ │ │ ┌──────────────────┐
┌──────────────────┐ │ │ │ Task Input │ ──────────────────▶ │ Channel │ │ │ └──────────────────┘
└──────────────────┘ │ │ │ │ │ │ ▼ ▼ │ │ ┌──────────────────┐ ┌──────────────────┐ │ │ │ 10-Step Flow│
◀──────────────────▶ │ Main Agent │ │ │ └──────────────────┘ └──────────────────┘ │ │ │ │ │ │ ▼ ▼ │ │
┌──────────────────┐ ┌──────────────────┐ │ │ │ Sub-agents │ ◀──────────────────▶ │ Subagent │ │ │
└──────────────────┘ └──────────────────┘ │ │ │ │ │ │ ▼ ▼ │ │ ┌──────────────────┐ ┌──────────────────┐
│ │ │ Memory │ ◀──────────────────▶ │ memory/ dir │ │ │ └──────────────────┘ └──────────────────┘ │ │ │
└──────────────────────────────────────────────────────────────────────────────────────────┘ `
```

---

2. DEERFLOW机制 - 复杂任务处理框架

2.1 概述

DEERFLOW（Distributed Execution Engine for Robust Workflow Orchestration）是一个专门处理复杂任务的分布式执行框架。它通过自动任务拆解、并行执行和沙箱隔离，实现对复杂任务的可靠处理。

2.2 5大核心模块

```
`  ┌──────────────────────────────────────────────────────────────────────────────────────────┐ │
DEERFLOW 架 构 全 景 图 │
├──────────────────────────────────────────────────────────────────────────────────────────┤ │ │ │
┌──────────────────────────────────────────────────────────────────────────────────────────┐ │ │ │ 任务编排层 │ │ │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ │ │ │ │ Module 1│ │ Module 2│ │ Module
3│ │ Module 4│ │ │ │ │ │ Planner │ │ Split │ │ Parallel│ │ Aggregate│ │ │ │ │ └──────┬───────┘ └──────┬───────┘
└──────┬───────┘ └──────┬───────┘ │ │ │
└──────┴──────┴──────┴──────┴──────┴──────┘ │ │ │ │ │ │ │ │ ▼ ▼ ▼
▼ │ │ ┌──────────────────────────────────────────────────────────────────────────────────────┐ │ │ │ 执行隔离层
(Module 5) │ │ │ │ ┌──────────────────────────────────────────────────────────────────────────┐ │ │ │ │ │ Docker Sandbox │ │ │ │ │ ┌──────┐
┌──────┐ ┌──────┐ │ │ │ │ │ │Sub-1│ │Sub-2│ │Sub-N│ │ │ │ │ │ │ └──────┘ └──────┘ └──────┘ │ │ │ │
│ └──────────────────────────────────────────────────────────────────────────────────────────┘ │ │ │
└──────────────────────────────────────────────────────────────────────────────────────────┘ │ │ │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘ `
```

模块详解

模块	名称	功能	关键技术
M1	任务规划器 (Planner)	分析任务复杂度，选择执行策略	LLM推理、复杂度评估
M2	任务拆分器 (Splitter)	将复杂任务分解为可并行子任务	依赖图构建、DAG生成
M3	并行执行器 (Parallel Executor)	调度多个子任务并行执行	线程池、协程调度
M4	结果聚合器 (Aggregator)	合并子任务结果，确保一致性	结果去重、冲突解决
M5	沙箱隔离器 (Sandbox Isolator)	提供隔离执行环境	Docker、资源限制

2.3 自动拆解与并行执行

拆解策略

` 任务复杂度评估 │ ├── 简单任务 (单步) ──→ 直接执行 │ ├── 中等任务 (2-5步) ──→ 串行执行 │ └── 复杂任务 (5+步) ──→ 并行拆解 │ ▼ ┌─────────────────┐ │ 构建依赖图 │ │ 识别并行路径 │ └─────────┬─────────┘ │ ┌─────────┴─────────┐ ▼ ▼ 串行依赖链 并行任务组 [A→B→C] [D,E,F,G] │ │ └─────────┬─────────┘ ▼ ┌─────────────────┐ │ 调度执行 │ │ 资源优化 │ └─────────────────┘ `

并行执行示例

假设有一个复杂的数据分析任务：

` 原始任务: "分析过去一年的销售数据，生成周报和月报"

拆解结果: ┌──┐ │ Task A: 数据清洗 (依赖: 无) │ │ └── 输出: clean_data.csv │ │ │ Task B: 周报生成 (依赖: A) │ │ └── 输入: clean_data.csv │ │ └── 输出: weekly_report.pdf │ │ │ Task C: 月报生成 (依赖: A) │ │ └── 输入: clean_data.csv │ │ └── 输出: monthly_report.pdf │ │ │ Task D: 汇总邮件 (依赖: B, C) │ │ └── 输入: weekly_report.pdf, monthly_report.pdf │ │ └── 输出: summary_email │ └──┘

执行顺序: A → (B ∥ C) → D 执行时间: T(A) + max(T(B), T(C)) + T(D) ≈ 串行的40% `

2.4 Docker沙箱隔离

沙箱架构

` ┌──┐ │ Host System │ │ ┌──┐ │ │ Docker Engine │ │ │ │ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ │ │ │ Sandbox 1 │ │ Sandbox 2 │ │ Sandbox N │ │ │ │ │ (Subtask) │ │ (Subtask) │ │ (Subtask) │ │ │ │ │ │ │ │ • 文件系统 │ │ • 文件系统 │ │ • 文件系统 │ │ │ │ │ • 网络隔离 │ │ • 网络隔离 │ │ • 网络隔离 │ │ │ │ │ • 资源限制 │ │ • 资源限制 │ │ • 资源限制 │ │ │ │ │ • 时间限制 │ │ • 时间限制 │ │ • 时间限制 │ │ │ │ └──────────────┘ └──────────────┘ └──────────────┘ │ │ │ │ │ │ └──┘ │ │ │ │ ▼ │ │

```
┌──────────────
```

层级	名称	存储内容	更新频率	保留时间
L0	事件层 (Event)	原始交互、操作记录、系统日志	实时	30天
L1	知识层 (Knowledge)	项目总结、任务经验、技术笔记	每日	永久
L2	智慧层 (Wisdom)	通用原则、模式识别、战略洞察	每周	永久

各层存储示例

L0层 - 原始事件 (`memory/2026-03-29.md`) ` markdown

今日事件

- 09:30 用户请求生成架构报告
- 09:35 创建报告目录结构
- 09:40 开始编写HARNESS章节
- 10:15 完成DEERFLOW模块图

...

`

L1层 - 项目知识 (`memory/system/architecture.md`) ` markdown

架构报告生成规范

- 必须包含5大核心模块
- 使用Mermaid语法绘制架构图
- 输出路径: energy_storage/data/reports/

经验教训

- PDF生成时使用fpdf2库
- 中文支持需要加载中文字体

`

L2层 - 通用智慧 (`MEMORY.md`) ` markdown

文档生成原则

- 分层架构优于扁平结构

- 图表比文字更直观
- 示例代码加速理解

系统思考

- 每个新功能都应考虑可观测性
- 复杂性应隐藏在模块内部

`

3.3 检索审计系统

检索流程

` 用户查询 │ ▼ ┌──────────────────────┐ │关键词提取 │ │意图理解 │ └──────────┬──────────┬──────────┘ │ ┌──────┴──────┐ ▼ ▼ L2检索 L1检索 L0检索 (语义) (关键词) (全文) │ │ │ └──────┬──────┘ │ ▼ │ ┌──────────────────┐ │ │ 相关性排序 │ │ │ 时间衰减 │ │ └──────────┬──────────┘ │ │ │ ▼ ▼ ┌──────────────────────────┐ │结果融合 │ │(去重 + 权重) │ └──────────────┬──────────────┘ │ ▼ ┌──────────────────────────┐ │返回给用户 │ │+ 记录审计日志 │ └──────────────────────────┘ `

审计日志格式

` json { "timestamp": "2026-03-29T09:30:00Z", "query": "HARNESS架构", "layers_accessed": ["L1", "L2"], "files_read": [ "memory/system/architecture.md", "MEMORY.md" ], "results_count": 5, "confidence_score": 0.92 } `

3.4 与Harness的集成

` ┌──┐ │ Harness + OPENVIKING 集成架构 │ ├──┤ │ │ │ Harness Workflow OPENVIKING Memory │ │ ──────────────────── ──────────────────── │ │ │ │ ① 任务接入 ──────────────────────────────▶ 写入L0层 (实时) │ │ │ │ │ ② 意图解析 ──────────────────────────────▶ 检索L1/L2层 │ │ │ (获取历史上下文) │ │ │ │ │ ③-⑧ 执行过程 ──────────────────────────▶ 更新L0层 │ │ │ (记录执行日志) │ │ │ │ │ ⑨ 质量校验 ──────────────────────────▶ 检索L1层 │ │ │ (对比质量标准) │ │ │ │ │ ⑩ 数据沉淀 ──────────────────────────▶ 更新L1/L2层 │ │ (经验提取 + 摘要生成) │ │ │ └──┘ `

---

4. 任务完成全链条机制

4.1 完整流程概览

`

┌──

```
┌──────────────
```

故障 ──→ 优雅降级 `

自愈策略

策略	适用场景	实现方式
重试	瞬态故障	指数退避，最多5次
降级	非核心功能失败	切换备用方案
熔断	依赖服务异常	暂停调用，返回缓存
回滚	数据错误	恢复到上一稳定状态
隔离	子任务失败	终止子任务，不影响整体

自愈流程示例

```
API调用失败
    │
    ▼
┌──────────────────────┐
│ 1. 记录错误日志      │
│ (L0层)               │
└──────────────────────┘
    │
    ▼
┌──────────────────────┐
│ 2. 判断错误类型      │
│ 可重试?              │
└──────────────────────┘
    │
    ┌────────┴────────┐
    ▼                 ▼
  是                否
    │                 │
    ▼                 ▼
┌──────────┐      ┌──────────────────┐
│ 3. 重试  │      │ 3. 启动人工介入  │
│ 指数退避 │      │ 发送通知         │
└──────────┘      └──────────────────┘
    │
    ▼
┌──────────────────────┐
│ 4. 重试成功?         │
└──────────────────────┘
    │
    ┌────────┴────────┐
    ▼                 ▼
  是                否
    │                 │
    ▼                 ▼
┌──────────┐      ┌──────────────────┐
│ 5. 继续  │      │ 5. 标记失败      │
│          │      │ 执行             │
│          │      │ 触发告警         │
└──────────┘      └──────────────────┘
```

---

5. 已打通的渠道集成

5.1 渠道概览

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              渠道集成总览                                │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│                              ┌──────────────┐                            │
│                              │   Agent      │                            │
│                              │              │                            │
│                              │   Core       │                            │
│                              │              │                            │
│                              └──────────────┘                            │
│                                    │                                     │
│                                    ▼                                     │
│                                    ▼                                     │
│                                    ▼
```

属性	配置
协议	SMTP (发送) / IMAP (接收)
服务器	smtp.qq.com / imap.qq.com
端口	465 (SSL) / 993 (SSL)
发件邮箱	lei547276643@qq.com
认证方式	授权码

功能支持:

- □ 发送文本邮件
- □ 发送HTML邮件
- □ 多附件支持
- □ 抄送/密送

`python

使用示例

send_email(to="recipient@qq.com", subject="系统通知", body="任务已完成", attachments=["report.pdf"]) `

5.2.2 飞书 Bitable

属性	配置
API类型	Feishu Open API
认证方式	App ID + App Secret
数据格式	JSON
权限	读/写/更新

功能支持:

- □ 创建/更新记录
- □ 批量导入数据
- □ 字段类型自动映射
- □ 多表关联

`python

使用示例

feishu_bitable.create_record(app_token="xxx", table_id="xxx", fields={ "任务名称": "数据同步", "状态": "已完成", "完成时间": "2026-03-29" }) `

5.2.3 IMA笔记

属性	配置
平台	腾讯IMA (Intelligent Memory Assistant)
同步方式	API调用
内容格式	Markdown支持

功能支持:

- □ 创建知识库笔记
- □ 自动分类标签
- □ 全文搜索
- □ 多设备同步

5.2.4 GitHub自动同步

属性	配置
API	GitHub REST API
认证	Personal Access Token
仓库	指定工作目录

功能支持:

- □ 自动提交代码
- □ 创建Pull Request
- □ Issue自动创建
- □ Release发布

`python

使用示例

github.sync_to_repo(repo="user/repo", branch="main", files=["src/", "docs/"], commit_msg="Auto-sync: 2026-03-29") `

5.2.5 企业微信Webhook

属性	配置
接入方式	群机器人Webhook
消息格式	Markdown / 文本 / 卡片
安全验证	可选签名验证

功能支持:

- □ 群消息推送
- □ @指定成员
- □ 图文消息
- □ 文件上传

`python

使用示例

wechat_webhook.send(webhook_url="https://qyapi.weixin.qq.com/cgi-bin/webhook/send?key=xxx", msg_type="markdown", content="## 任务完成\n报告已生成") `

5.3 集成架构图

```
` ┌──────────────
```

```
└──────────────┘ └──────────────┘ │ │ │
└──────────────────────────────────────────────────────────────────────┘ `
```

5.4 渠道配置汇总

渠道	状态	使用场景	优先级
QQ邮箱	□ 已打通	正式报告发送、通知	高
飞书Bitable	□ 已打通	数据存储、任务追踪	高
IMA笔记	□ 已打通	知识沉淀、文档管理	中
GitHub	□ 已打通	代码同步、版本管理	中
企微Webhook	□ 已打通	即时通知、群消息	高

---

6. 总结与展望

6.1 核心优势

维度	优势
可靠性	HARNESS 10步流程确保任务可靠完成
效率	DEERFLOW并行执行提升40%+效率
智能	OPENVIKING记忆系统实现长期上下文保持
安全	多层漂移检测 + 沙箱隔离
集成	5大渠道打通，实现端到端自动化

6.2 技术栈总结

```
`  ┌──────────────────────────────────────────────────────────────────────┐ │ 完
整                    技                    术                    栈                    │
├──────────────────────────────────────────────────────────────────────┤ │ │ │ 应
用层: OpenClaw / Agent系统 │ │ │ │ 框架层: HARNESS + DEERFLOW + OPENVIKING │ │ │ │ 执行层: Docker
Sandbox / Subagent调度 │ │ │ │ 存储层: Memory Files / Feishu Bitable / GitHub │ │ │ │ 通信层: SMTP / IMAP /
Webhook / REST API │ │ │
└──────────────────────────────────────────────────────────────────────┘ `
```

6.3 后续规划

- **性能优化**: 引入向量数据库加速记忆检索
- **扩展性**: 支持更多渠道接入（钉钉、Slack等）
- **智能化**: 基于历史数据预测任务风险
- **可视化**: 开发任务执行实时监控面板

文档结束

生成时间: 2026-03-29 03:09 GMT+8